Java Web Development Environment

SCSE 3203 Special Topics
Week 02

Learning Plan for Week 02

IN CLASS · 4 HOURS

- Concepts explained through lecture slides
- Installation and implementation through guided lab
- Minimum: a working server-rendered page

AFTER CLASS · SDL

- Independent extension activity
- Concepts, code reasoning and diagnostics
- Interactive assessment and reflection

Learning outcomes

- Explain how Java, the JDK, Maven, Spring Boot and Tomcat work together
- Configure and verify a Java development environment
- Generate, run and package a Spring Boot Maven project
- Render HTML using a Spring MVC controller and Thymeleaf
- Apply basic Git/GitHub and troubleshoot common issues

What we have—and what next

INITIAL STATE

- Windows computer and web browser
- VS Code installed but unconfigured
- Internet access and installation permissions

NOT YET INSTALLED

- JDK or Java extension packs
- Global Maven, Git or standalone Tomcat
- Spring Boot project or dependencies

Zero-to-working-webpage steps

1. Install JDK 21
2. Configure VS Code
3. Generate Maven project
4. Run Spring Boot
5. Create MVC + HTML
6. Verify browser output

GitHub and executable JAR can be completed after class if installation takes longer.

What belongs to this Week 02—and what does not

- **In scope**

  Toolchain, Maven, embedded Tomcat, simple server-rendered HTML, basic Git

- **Guided exposure**

  @Controller, @GetMapping, logical view names; no Spring internals

- **Deferred to later weeks**

  Dependency injection, dynamic Thymeleaf forms, persistence, security, REST, JSON and Fetch

Each tool has a distinct responsibility

Technology	Purpose
Java / JDK	Programming, compilation and runtime tools
VS Code	Editor, extension host, terminal and debugger
Maven	Dependencies, compilation, testing, packaging
Spring Boot	Application bootstrap and conventional configuration
Embedded Tomcat	Accepts and processes HTTP connections
Thymeleaf	Server-side HTML template rendering
Git / GitHub	Version history / collaborative hosting

Build-time workflow

1. Write Java sources
2. Declare dependencies
3. Resolve via Maven
4. Compile to bytecode
5. Run tests
6. Package executable JAR

Build output is not the same as source code; generated target/ content normally stays out of Git.

Runtime page-rendering workflow

1. Browser GET /
2. Embedded Tomcat
3. Spring MVC routing
4. HomeController
5. Thymeleaf template
6. HTML to browser

The browser receives HTML generated on the server—not a JSON response.

JDK, compiler and JVM

JDK

- Development kit including javac and java
- Compiles .java source to .class bytecode
- Selected baseline: Java 21 LTS

JVM / EXECUTION

- JVM executes Java bytecode
- Runtime manages loading and execution
- Editor is not necessary to run a packaged app

Verify the JDK installation

```
java -version
javac -version
where.exe java
where.exe javac
```

Checkpoint JDK-PASS: both Java commands succeed and report the intended JDK 21 installation.

VS Code is the development environment—not the JDK

- Install Extension Pack for Java
- Install Spring Boot Extension Pack
- Open integrated terminal and ensure JDK is detected
- Allow Java project import and Maven downloads to complete
- Use editor diagnostics and the terminal together

Why Maven matters

WITHOUT MAVEN

- Manual retrieval of multiple JAR files
- Difficult dependency compatibility
- Nonstandard compilation and packaging
- Poorly reproducible builds

WITH MAVEN

- Dependencies declared in pom.xml
- Transitive dependency resolution
- Standard compile / test / package lifecycle
- Reproducible project configuration

The identity of a Maven project

```
<groupId>my.utm.scse3203</groupId>
<artifactId>week02-demo</artifactId>
<version>0.0.1-SNAPSHOT</version>
```

Coordinates identify a Maven artifact. The generated pom.xml also declares starters and build plugins.

Read the Maven POM—not every XML tag

POM element	What students must understand
groupId	Logical namespace or organisation
artifactId	Name of the generated artifact
version	Current development / release version
dependencies	Required libraries and starters
parent	Shared build and dependency management
build / plugins	Tools for packaging and execution

Maven Wrapper commands on Windows

```
.\mvnw.cmd -version
.\mvnw.cmd compile
.\mvnw.cmd test
.\mvnw.cmd package
.\mvnw.cmd spring-boot:run
```

Maven Wrapper avoids a separate global Maven installation. spring-boot:run is a plugin goal.

Spring Framework vs Spring Boot

SPRING FRAMEWORK

- Application infrastructure and web MVC
- Flexible configuration model
- Core concepts such as components and DI
- Supports many application architectures

SPRING BOOT

- Built on the Spring Framework
- Starter dependencies and conventions
- Auto-configuration of common facilities
- Easy startup with embedded server

Spring Boot starter dependencies

- **Spring Web MVC**

  Provides servlet-based Spring MVC and embedded Tomcat support

- **Thymeleaf**

  Adds the server-side HTML template engine

- **For Spring Boot 4.x**

  Use instructor-verified spring-boot-starter-webmvc and spring-boot-starter-thymeleaf

Embedded versus external Tomcat

TRADITIONAL DEPLOYMENT

- Separate servlet container installation
- WAR typically deployed to configured server
- Server and application managed separately

OUR WEEK 02 SETUP

- No standalone Tomcat installation
- Spring Boot provides embedded Tomcat
- Run through Maven or executable JAR

What auto-configuration means

- Spring Boot examines selected dependencies and application settings
- Many common components are configured automatically
- An application can start an embedded web server without manual server setup
- The mechanism behind auto-configuration is not part of Week 02

From Spring Initializr to a local server

1. Choose Maven + Java
2. Use pinned Boot 4.x
3. Select Java 21
4. Add MVC + Thymeleaf
5. Generate ZIP
6. Open root in VS Code

Pin one version and consistent starter names for all students.

Spring Initializr: shared class configuration

Field	Selected value
Project / Language	Maven / Java
Group	my.utm.scse3203
Artifact	week02-demo
Base package	my.utm.scse3203.week02
Packaging / Java	JAR / Java 21
Dependencies	Spring Web MVC + Thymeleaf

Understand the application bootstrap

```
@SpringBootApplication
public class Week02DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            Week02DemoApplication.class, args);
    }
}
```

The main() method launches Spring Boot. Detailed component scanning and IoC are later topics.

Anatomy of the generated project

Location	Role
pom.xml	Maven metadata, dependencies and build
src/main/java	Application classes and controller
src/main/resources/templates	HTML templates for server rendering
src/main/resources/static	Static assets such as CSS/images
application.properties	Externalised application configuration
src/test/java	Automated test sources
mvnw.cmd / .mvn	Project Maven Wrapper

Templates are not the same as static files

TEMPLATES

- src/main/resources/templates
- Resolved through the view-rendering process
- Week 02: index.html and about.html

STATIC RESOURCES

- src/main/resources/static
- Normally served directly as files
- Suitable for CSS, images and JS assets

How Spring MVC renders the home page

1. GET request: /
2. Routing finds mapping
3. @Controller method
4. Returns "index"
5. Thymeleaf loads index.html
6. Browser displays HTML

This is a server-rendered page, not a REST endpoint.

A minimal MVC controller

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "index";
    }
}
```

"index" is a logical view name—not text to display. Add the correct package and imports in the lab.

First Thymeleaf-compatible HTML template

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8">
<title>SCSE 3203</title></head>
<body>
  <h1>Java Web Development</h1>
  <p>My first server-rendered page.</p>
</body>
</html>
```

Save as src/main/resources/templates/index.html. Dynamic expressions are taught later.

What happens when user press Run?

OBSERVE IN TERMINAL

- Spring Boot application startup
- Embedded Tomcat listening on port 8080
- Application startup messages
- Errors if compilation or server fails

OBSERVE IN BROWSER

- http://localhost:8080/
- HTML homepage from Thymeleaf
- 404 before a mapping exists
- Page source contains HTML response

Add a second page

```
@GetMapping("/about")
public String about() {
    return "about";
}

// Template:
// src/main/resources/templates/about.html
```

The controller maps a URL; Thymeleaf resolves the corresponding template.

Application properties and configuration

```
spring.application.name=week02-demo
server.port=8081

# Restart application, then open:
# http://localhost:8081/
```

Changing the port illustrates configuration outside source code. Restore 8080 after testing.

Build and run without VS Code

```
.\mvnw.cmd clean package

java -jar target\week02-demo-0.0.1-SNAPSHOT.jar
```

Use the actual generated JAR filename. The editor is not required to execute the packaged application.

Git workflow from local work to GitHub

1. Edit files
2. Inspect git status
3. Stage changes
4. Commit locally
5. Push to remote
6. Verify on GitHub

Do not commit target/, secrets, access tokens or personal credentials.

Git vs GitHub

GIT

- Distributed version-control system
- Stores commits and branch history locally
- Can work without an internet connection

GITHUB

- Remote repository and collaboration service
- Hosts shared project history
- Enables code review, pull requests and automation

Essential Git commands

```
git --version
git status
git add .
git commit -m "Add server-rendered home page"
git push
```

Repository authentication and GitHub Classroom onboarding belong in the guided lab workflow.

CI/CD: recognition, not implementation

CONTINUOUS INTEGRATION

- Push triggers automated build checks
- Maven compiles and runs tests
- Failures become visible to the team

- One conceptual demonstration is sufficient
- No GitHub Actions YAML authoring required
- More CI/CD work deferred to Week 14

Troubleshooting through evidence

Symptom	Likely starting point
java / javac missing	PATH, JAVA_HOME, JDK installation
Wrong Java version	where.exe java; active JDK selection
Dependency download failure	Maven log; network/proxy/repository
Port 8080 in use	Conflicting application or configured port
Page shows 404	Controller mapping and requested path
Template cannot be resolved	Logical view name and templates folder
git push rejected	Remote URL, permissions, authentication

Evidence-based diagnosis

1. **Reproduce failure**
2. **Read first relevant error**
3. **Classify failure**
4. **Form likely hypothesis**
5. **Change one variable**
6. **Rerun and document**

Do not randomly reinstall every development tool when a targeted explanation is available.

Completion criteria for week 02

MINIMUM / MUST COMPLETE

- Java 21 and editor extensions ready
- Maven Wrapper executes in project
- Spring Boot starts successfully
- Server-rendered homepage works

TARGET / MAY CONTINUE AS SDL

- About and Profile pages
- JAR packaging and execution
- GitHub remote repository verification
- Independent troubleshooting extension

Project briefing and readiness for Week 03

- Confirm team members and a temporary coordinator
- Identify an initial Java web application idea
- Agree on communications and GitHub collaboration approach
- Keep the Week 02 starter project for future work
- Detailed SDLC planning and requirements start in Weeks 03–05

After-class SDL assessment

ASSESSMENT JOURNEY

- Development readiness checkpoints
- Conceptual MCQs and code interpretation
- Diagnostics and practical extension
- Reflection and targeted feedback

IMPORTANT LIMITATIONS

- Completion is outside the 4 contact hours
- MCQ score alone does not prove tool proficiency
- Practical evidence from running project/GitHub
- Formative feedback before high-stakes grading

Key takeaways

- A Java web application is produced by an integrated development toolchain
- Maven manages dependencies and repeatable builds; Spring Boot runs the application
- Embedded Tomcat receives browser requests; a controller selects a view
- Thymeleaf renders HTML on the server before the browser displays the page
- Week 02 success = working toolchain + working local server-rendered page

Next: Guided laboratory

START WITH A CLEAN VS CODE INSTALLATION

Install → Verify → Generate → Run → Render → Commit

Use the separate Week 02 Lab Worksheet for every click, command, checkpoint and troubleshooting step.